



Motus

Jeu de mots en microservices — Rapport de projet

Applications Web orientées Services — M2 MIAGE SITN

Université Paris-Dauphine · Année 2025-2026

Réalisé par **Liya & Hongxiang**

Dépôt GitHub : github.com/Steven-1105/M2_SITN_Motus (branche `main`)

1. Lancement rapide du projet

Instructions détaillées (prérequis, dépannage, comptes de démonstration) dans le `README.md` à la racine du dépôt. Résumé :

```
git clone https://github.com/Steven-1105/M2_SITN_Motus.git
cd M2_SITN_Motus
docker compose up --build          # démarre MySQL + les 4 microservices (8081-8084)

cd frontend
python3 -m http.server 8090        # sert le frontend statique
# puis ouvrir http://localhost:8090
```

Compte administrateur de démonstration : identifiant `admin` , mot de passe `admin123` (accès au panneau d'administration). Douze comptes joueurs peuplent également le classement.

Un déploiement Kubernetes (Minikube) est aussi fourni dans `k8s/` , avec un script `deploy.sh` et un `README.md` dédié.

2. Choix techniques

Backend

- Java 26, Spring Boot 4
- Spring Data JPA + MySQL 8
- API REST (JSON) entre services
- Un microservice = une base de données dédiée

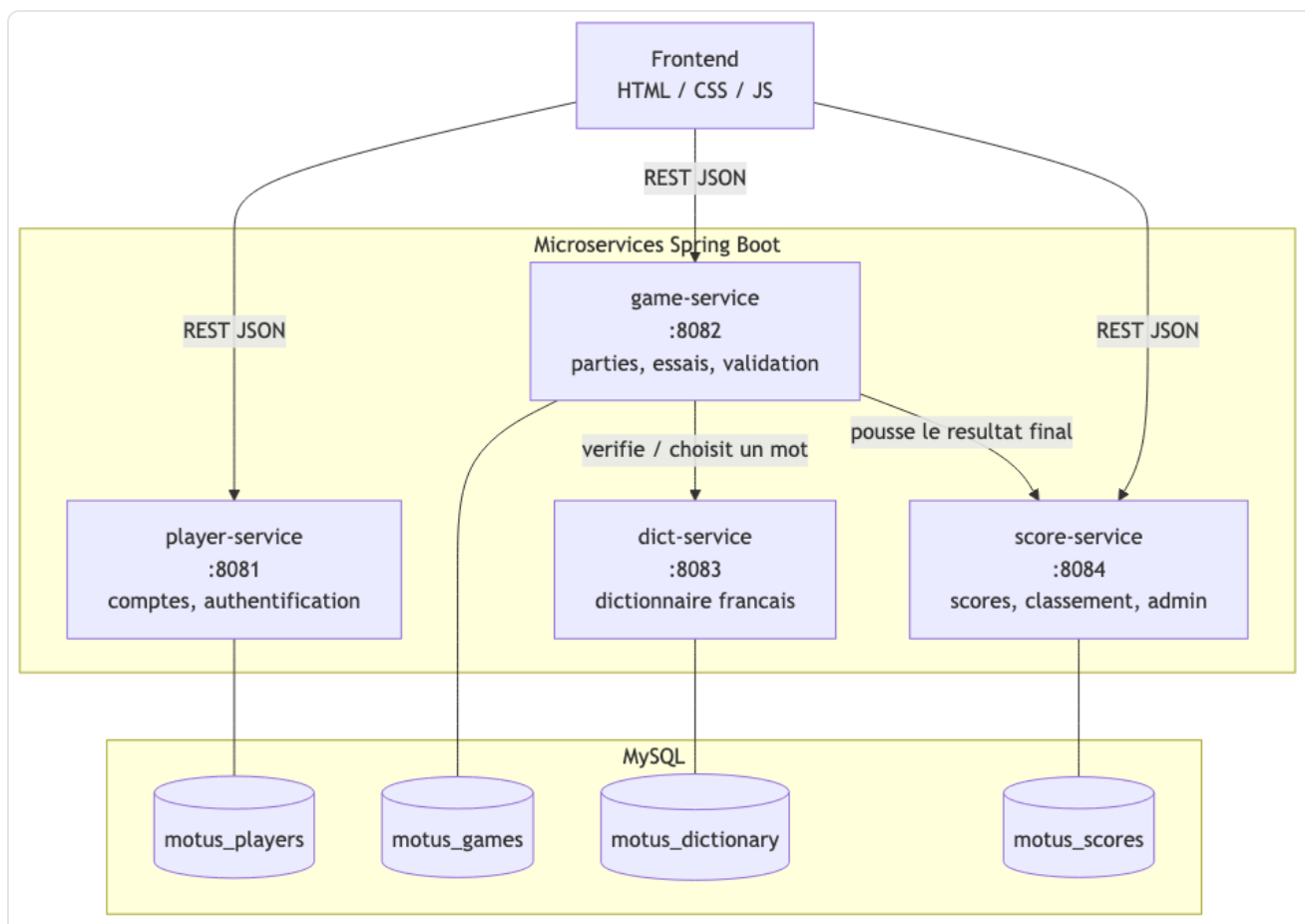
Frontend & infra

- HTML / CSS / JavaScript natif (sans framework)
- Docker + Docker Compose pour le développement local
- Kubernetes (Minikube) pour le déploiement
- Maven (un `pom.xml` par service)

Nous avons choisi une architecture en quatre microservices indépendants (`player-service` , `game-service` , `dict-service` , `score-service`) afin de séparer clairement les responsabilités métier : comptes/authentification, logique de jeu, dictionnaire, et scores/statistiques/administration. Chaque service peut être développé, testé et déployé indépendamment.

3. Architecture des microservices

Le frontend communique en REST/JSON avec les trois services exposés publiquement. `game-service` orchestre la partie : il interroge `dict-service` pour choisir/valider un mot, puis pousse le résultat final vers `score-service` une fois la partie terminée.

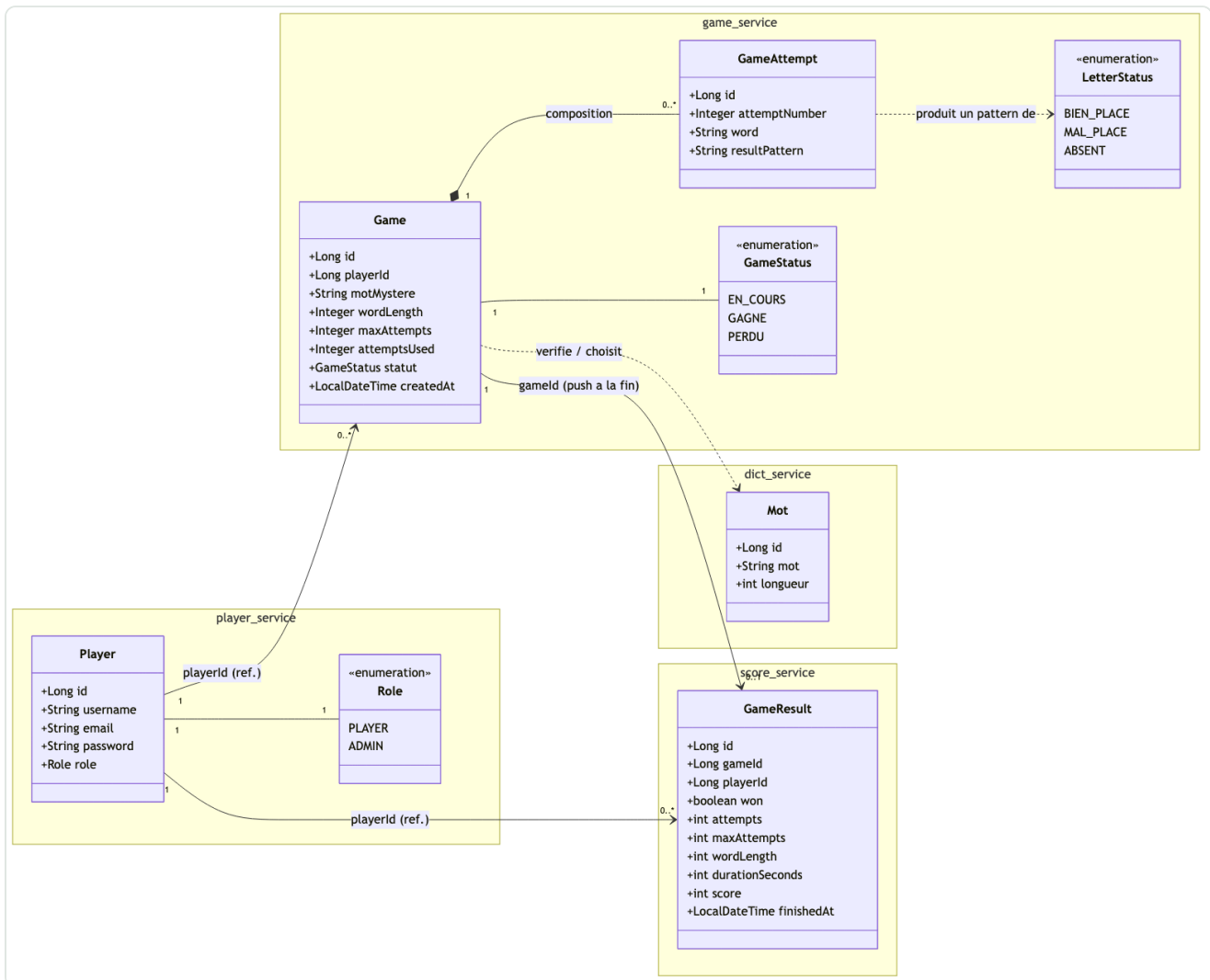


Architecture générale : frontend, 4 microservices Spring Boot, une base MySQL par service.

Service	Port	Responsabilité
player-service	8081	Inscription, connexion, gestion des comptes joueurs (rôles PLAYER / ADMIN)
game-service	8082	Création de partie, soumission des essais, feedback lettre par lettre
dict-service	8083	Dictionnaire français : mot aléatoire par longueur, validation d'un mot
score-service	8084	Historique des scores, statistiques, classement global, recherche pour l'administration

4. Diagramme de classes

Le modèle métier est réparti entre les quatre microservices ; chaque service possède ses propres entités JPA et sa propre base. Il n'y a donc pas de clé étrangère entre services : les liens (`playerId` , `gameId`) sont de simples références par identifiant, résolues par appel REST au niveau applicatif.



Entités métier par microservice et leurs relations logiques.

5. Bilan du projet

Ce que nous avons apprécié

Travailler sur une vraie architecture microservices nous a permis de nous répartir clairement le travail : chacun a pu avancer sur ses services (player/game d'un côté, dict/score de l'autre) sans se marcher dessus, en se synchronisant via une branche `integration` commune. La logique de jeu façon Motus (feedback lettre par lettre, gestion des essais) a aussi été un exercice concret et amusant à implémenter.

Ce qui nous a moins plu

La coordination entre services au démarrage a demandé plus de configuration que prévu (variables d'environnement, noms de service dans le réseau Docker, ordre de démarrage avec les `healthcheck` MySQL). Gérer un dictionnaire français de plus de 130 000 mots a aussi révélé des cas particuliers pénibles à filtrer (pluriels, mots archaïques, formes conjuguées).

Ce que nous avons appris

- Concevoir une API REST claire entre services indépendants, avec des contrats stables (DTO) plutôt que du partage de code.
- Isoler les données par service (une base par microservice) plutôt qu'une base partagée.
- Containeriser une application multi-services avec Docker Compose, puis la déployer sur Kubernetes (Minikube).
- Travailler en équipe sur un même dépôt Git avec des branches par fonctionnalité et une branche d'intégration commune.

Difficultés rencontrées

- Synchroniser le travail des deux membres de l'équipe sur des services qui communiquent entre eux (contrats d'API à stabiliser tôt).
- Nettoyer un dictionnaire de mots réel (doublons, formes non pertinentes) pour que le jeu reste cohérent et juste.
- Ajouter tardivement la fonctionnalité d'administration sans casser l'existant, en réutilisant un endpoint déjà exposé plutôt qu'en développant une nouvelle API.
- Respecter le calendrier du projet en parallèle des autres cours du semestre.